

Software Maintenance

- Software maintenance refers to modifying and updating software after delivery to correct faults, improve performance, or adapt to changes.

Types of Software Maintenance

- 1. Corrective
- 2. Adaptive
- 3. Perfective
- 4. Preventive

Corrective Maintenance

- **1. Corrective Maintenance**

- **Definition:**

Corrective maintenance is performed to **fix errors or bugs** found in the software after it has been delivered.

- **Purpose:**

To make the software work correctly by removing defects.

- **Example:**

A mobile app crashes when a user clicks the login button.

Developers fix the bug and release an update.

Adaptive Maintenance

- **2. Adaptive Maintenance**

- **Definition:**

Adaptive maintenance is done to **modify the software so that it works in a new or changed environment.**

- **Purpose:**

To keep the software compatible with new technologies, operating systems, or hardware.

- **Example:**

A software application is updated to work on a new version of Android or Windows.

Perfective Maintenance

- **3. Perfective Maintenance**

- **Definition:**

Perfective maintenance is performed to **improve performance or add new features** based on user requirements.

- **Purpose:**

To enhance the system and make it more efficient and user-friendly.

- **Example:**

Adding a new feature like “dark mode” in an application or improving app speed.

Preventive Maintenance

- **4. Preventive Maintenance**

- **Definition:**

Preventive maintenance is done to **prevent future problems** and improve software maintainability.

- **Purpose:**

To reduce the chances of future errors and improve code quality.

- **Example:**

Refactoring code, removing unused variables, and optimizing performance to avoid future bugs.

Software Re-Engineering

- **Definition**
- **Software Re-Engineering** is the process of **modifying and improving an existing software system** to make it better, more efficient, and easier to maintain **without changing its basic functionality**.

Purpose of Re-Engineering

- Improve software quality
- Make system easier to maintain
- Reduce complexity
- Improve performance
- Update old technology
- Reduce future maintenance cost

Why Re-Engineering is Needed

- Software is built using old technology
- Code is complex and hard to understand
- Documentation is missing or poor
- System performance is slow
- Maintenance cost is high

Activities in Software Re-Engineering

- **1. Reverse Engineering**
 - Analyzing the existing system to understand its design and working.
- **2. Code Restructuring**
 - Improving and cleaning the existing code without changing functionality.
- **3. Data Restructuring**
 - Improving database structure and data organization.
- **4. Forward Engineering**
 - Rebuilding the system using improved design and modern technology.

Process of Software Re-Engineering

- Analyze the existing system
- Identify problems
- Improve system design
- Restructure code and data
- Re-develop the system
- Test and deploy

Example: Banking System

- A bank is using an old software system:
- Developed in outdated programming language
- Slow performance
- Difficult to maintain
- **Re-Engineering Steps:**
- Developers analyze the old system
- Clean and restructure the code
- Improve database design
- Upgrade to modern technologies (new UI, faster backend)
- **Result:**
- System becomes faster
- Easier to maintain
- Better user experience

Advantages

- Lower cost than developing a new system
- Improves performance
- Extends software life
- Easier maintenance

Disadvantages

- Time-consuming
- Difficult if old system is poorly designed
- May not completely solve all issues

Reverse Engineering

- **Definition**
- Reverse Engineering is the process of **analyzing an existing software system to understand its design, structure, and functionality** without having its original documentation.

Objectives of Reverse Engineering

- Understand old or unknown software
- Recover missing documentation
- Analyze system design and structure
- Identify errors or weaknesses
- Support maintenance and re-engineering

Why Reverse Engineering is Needed

- Original documentation is missing
- Software is very old and complex
- Need to upgrade or modify the system
- To understand competitor software
- For debugging and maintenance

Process of Reverse Engineering

- **Input:** Existing software (source code or executable)
- **Code Analysis:** Study the code structure
- **Data Analysis:** Understand data flow and database
- **Design Recovery:** Recreate design and architecture
- **Documentation:** Prepare new documentation

Types of Reverse Engineering

- **1. Code Reverse Engineering**
 - Understanding program code and logic.
- **2. Design Reverse Engineering**
 - Understanding system architecture and design.
- **3. Data Reverse Engineering**
 - Understanding data structures and database design.

Example: Old Software System

- A company has an old payroll system:
- Developed many years ago
- No documentation available
- Developers who created it are no longer available
- **Reverse Engineering Steps:**
- Developers analyze the existing source code
- Understand how salary is calculated
- Study database structure
- Recreate system design and flowcharts
- Prepare proper documentation
- **Result:**
- System becomes understandable
- Easy to maintain and modify
- Can be upgraded in future

Forward Engineering

- **Definition**
- Forward Engineering is the process of **developing a new software system from requirements and design specifications.**
- It means **building software from scratch** using proper planning, design, and coding.

Objectives of Forward Engineering

- Develop a new software system
- Convert requirements into working software
- Use modern technologies and tools
- Create efficient and high-quality systems

Why Forward Engineering is Needed

- When a completely new system is required
- When old system cannot be improved further
- When new features and technologies are needed
- When starting a project from the beginning

Process of Forward Engineering

- **Requirement Analysis**
Understand user needs and system requirements
- **System Design**
Create architecture, modules, and design structure
- **Implementation (Coding)**
Write code based on design
- **Testing**
Check for errors and ensure proper working
- **Deployment**
Release the software for users
- **Maintenance**
Update and fix issues after delivery

Example: E-Commerce Website

- A company wants to develop a new online shopping website.
- **Forward Engineering Steps:**
 - Collect requirements (login, cart, payment, orders)
 - Design system architecture and database
 - Develop frontend and backend code
 - Test features like payment and login
 - Deploy the website for users
- **Result:**
 - A fully functional new software system is created

Advantages

- Uses latest technology
- Better performance and design
- Easy to maintain (if properly designed)
- High-quality output

Disadvantages

- **Disadvantages**
- Time-consuming
- Expensive compared to modifying existing system
- Requires proper planning and resources

Types / Approaches of Forward Engineering

- Forward Engineering does not have fixed “types” like maintenance, but it can be carried out using different **approaches or methods**.

1. Structured Approach

- In this approach, software is developed in a **step-by-step structured manner**.
- **Features:**
 - Follows clear phases (analysis → design → coding)
 - Uses tools like flowcharts and DFDs
 - Easy to understand and manage
- **Example:**

Developing a banking system using data flow diagrams and structured design techniques.

2. Object-Oriented Approach

- In this approach, software is designed using **objects and classes**.
- **Features:**
 - High reusability
 - Better modularity
 - Easy maintenance
- **Example:**

Creating an e-commerce system with classes like *User*, *Product*, and *Order*.

3. Component-Based Development

- In this approach, software is built using **pre-developed components or modules.**
- **Features:**
 - Faster development
 - Less coding effort
 - Reusable components
- **Example:**

Using existing login modules or payment gateway APIs in an application.

4. Agile / Iterative Approach

- In this approach, software is developed in **small parts (iterations)** with continuous feedback.
- **Features:**
 - Flexible and adaptable
 - Quick updates
 - Continuous improvement
- **Example:**

Developing a mobile app step-by-step (first login, then dashboard, then features).

Feature	Re-Engineering	Reverse Engineering	Forward Engineering
Purpose	Improve system	Understand system	Build new system
Direction	Code → Improved Code	Code → Design	Design → Code
Output	Better version of same system	Documentation/design	New software
Use Case	Legacy system upgrade	Code analysis	New development

Reverse = Backward (Code → Design)

Forward = New Development (Design → Code)

Re-Engineering = Improve Existing System